Research question: "How can k-means clustering techniques uncover patterns among patients who are readmitted to the hospital based on their health and hospitalization data?"

In [1]:
```
#Importing libraries
import pandas as pd
import numpy as np
from scipy.stats import zscore
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

In [2]:
```
#Loading the dataset
medical_clean = pd.read_csv('medical_clean_D212.csv')
```

In [3]:
```
#Information about the dataset
medical_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   CaseOrder           10000 non-null  int64
 1   Customer_id         10000 non-null  object
 2   Interaction         10000 non-null  object
 3   UID                 10000 non-null  object
 4   City                10000 non-null  object
 5   State               10000 non-null  object
 6   County              10000 non-null  object
 7   Zip                 10000 non-null  int64
 8   Lat                 10000 non-null  float64
 9   Lng                 10000 non-null  float64
 10  Population          10000 non-null  int64
 11  Area                10000 non-null  object
 12  TimeZone            10000 non-null  object
 13  Job                 10000 non-null  object
 14  Children            10000 non-null  int64
 15  Age                 10000 non-null  int64
 16  Income              10000 non-null  float64
 17  Marital             10000 non-null  object
 18  Gender              10000 non-null  object
 19  ReAdmis             10000 non-null  object
 20  VitD_levels         10000 non-null  float64
 21  Doc_visits          10000 non-null  int64
 22  Full_meals_eaten    10000 non-null  int64
 23  vitD_supp           10000 non-null  int64
 24  Soft_drink          10000 non-null  object
 25  Initial_admin       10000 non-null  object
 26  HighBlood           10000 non-null  object
 27  Stroke              10000 non-null  object
 28  Complication_risk   10000 non-null  object
 29  Overweight          10000 non-null  object
 30  Arthritis           10000 non-null  object
 31  Diabetes            10000 non-null  object
 32  Hyperlipidemia      10000 non-null  object
 33  BackPain            10000 non-null  object
 34  Anxiety             10000 non-null  object
 35  Allergic_rhinitis   10000 non-null  object
 36  Reflux_esophagitis  10000 non-null  object
 37  Asthma              10000 non-null  object
 38  Services            10000 non-null  object
 39  Initial_days        10000 non-null  float64
 40  TotalCharge         10000 non-null  float64
 41  Additional_charges  10000 non-null  float64
 42  Item1               10000 non-null  int64
 43  Item2               10000 non-null  int64
 44  Item3               10000 non-null  int64
 45  Item4               10000 non-null  int64
 46  Item5               10000 non-null  int64
 47  Item6               10000 non-null  int64
 48  Item7               10000 non-null  int64
 49  Item8               10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

In [4]:

```
#Selecting variables to be used for the algorithm
variables = ['Initial_days', 'Doc_visits', 'Age', 'ReAdmis', 'Overweight']
```

```
df = medical_clean[variables].copy()
```

In [5]:
```
#Checking for duplicate values
print(df.duplicated())
```

```
0       False
1       False
2       False
3       False
4       False
        ...
9995    False
9996    False
9997    False
9998    False
9999    False
Length: 10000, dtype: bool
```

In [6]:
```
#Checking for missing values
print(df.isnull().sum())
```

```
Initial_days    0
Doc_visits      0
Age             0
ReAdmis         0
Overweight      0
dtype: int64
```

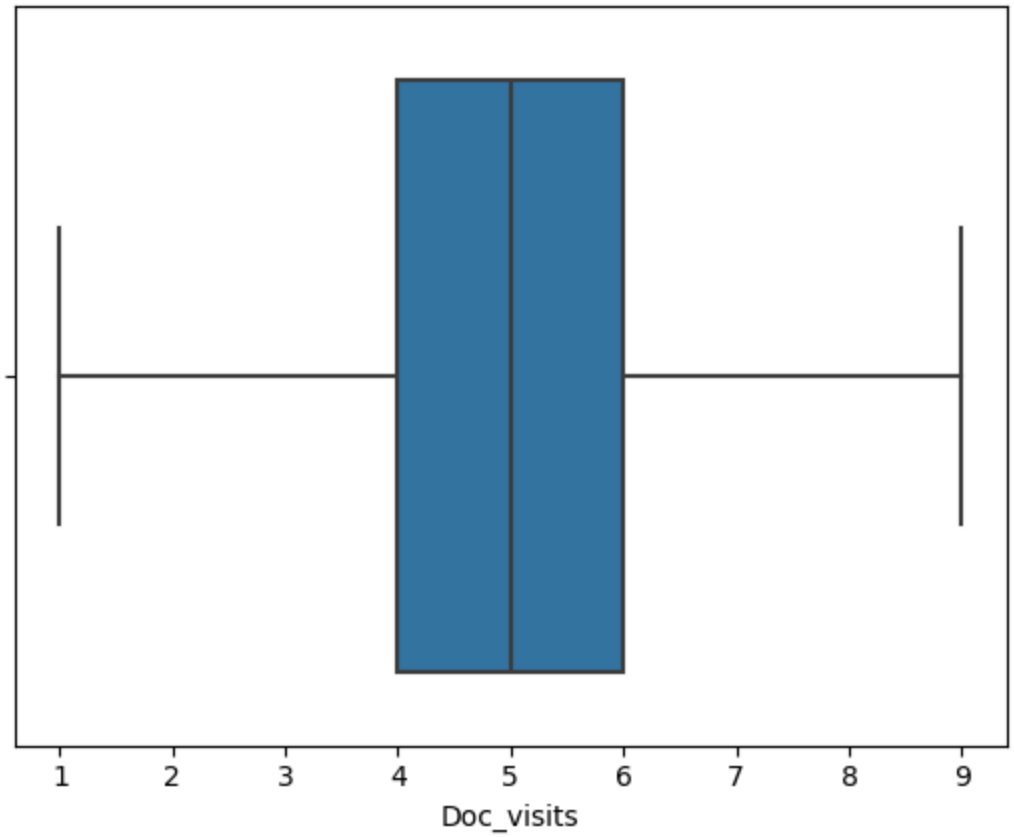
In [7]:
```
#List of numerical columns for checking outliers
numerical_columns = ['Initial_days', 'Doc_visits', 'Age']

#Identify outliers based on Z-scores
z_scores = np.abs(zscore(df[numerical_columns]))
z_score_outliers = (z_scores > 3).sum(axis=0)
print("Outliers based on Z-score method:")
print(z_score_outliers)
```

```
Outliers based on Z-score method:
Initial_days    0
Doc_visits      8
Age             0
dtype: int64
```

In [8]:
```
#Checking for outliers for Doc_visits
sns.boxplot(x = 'Doc_visits', data = df)
```

Out[8]:
```
<Axes: xlabel='Doc_visits'>
```



In [9]:
```
#Converting categorical variable Readmis to a numeric variable
df['ReAdmis'] = df['ReAdmis'].map({'Yes': 1, 'No': 0})
```

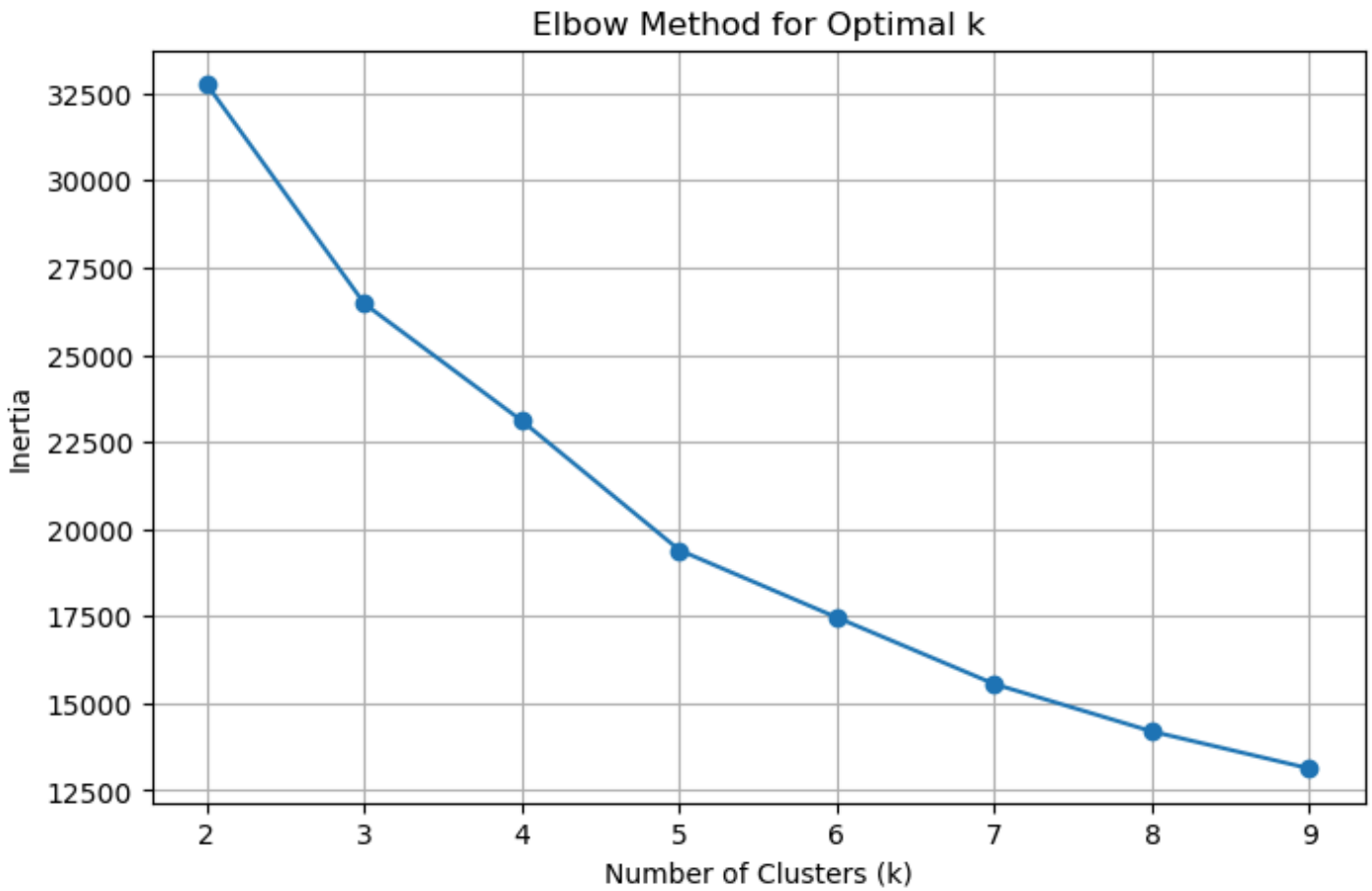
In [10]:
```
#Converting categorical variable Overweight to a numeric variable
df['Overweight'] = df['Overweight'].map({'Yes': 1, 'No': 0})
```

In [11]:
```
#Feature scaling
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
```

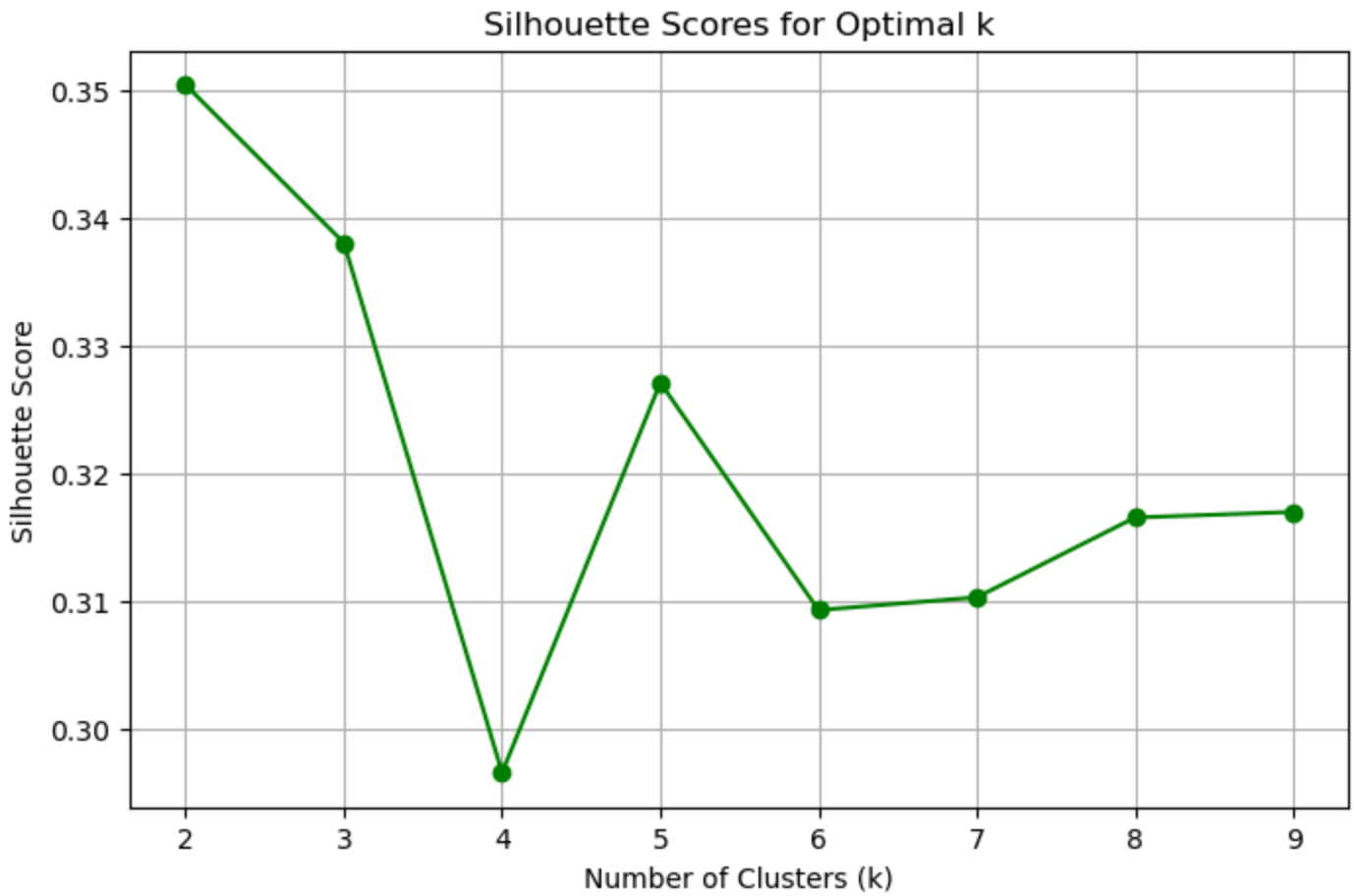
In [12]:
```
#Determining optimal number of clusters
inertia = []
silhouette_scores = []
k_range = range(2, 10)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(scaled_data)
    inertia.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(scaled_data, kmeans.labels_))
```

In [13]:
```
#Plot elbow method
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid()
plt.show()
```



In [14]:
```
#Plot silhouette scores
plt.figure(figsize=(8, 5))
plt.plot(k_range, silhouette_scores, marker='o', color='green')
plt.title('Silhouette Scores for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.grid()
plt.show()
```



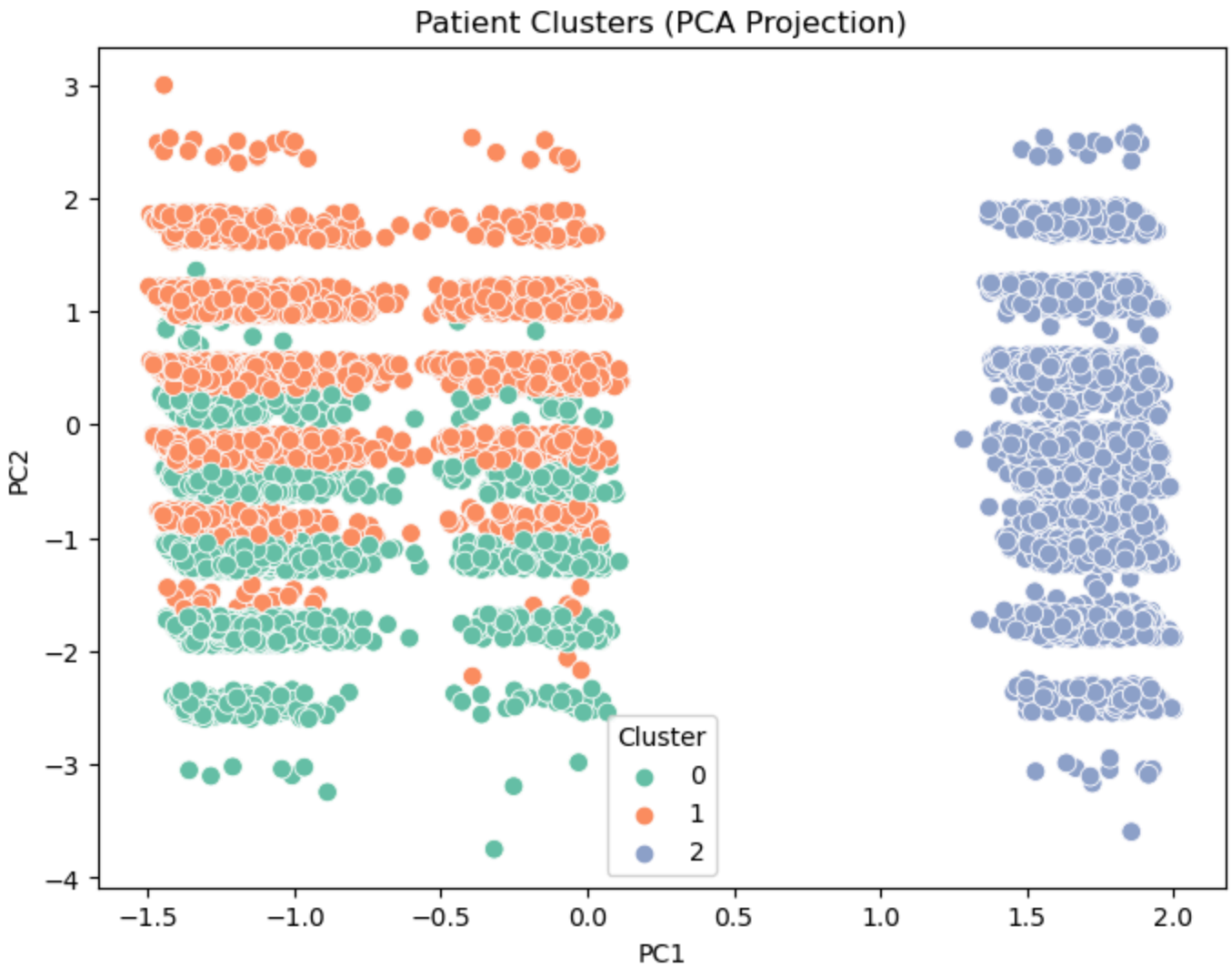
In [15]:
```
#Apply K-Means with optimal K
optimal_k = 3  # Select based on Elbow and Silhouette analysis
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
clusters = kmeans_final.fit_predict(scaled_data)
```

In [16]:
```
#Adding cluster labels to the dataset
df['Cluster'] = clusters
```

In [17]:
```
#Cluster analysis
cluster_summary = df.groupby('Cluster').mean()
print("Cluster Summary:")
print(cluster_summary)
```

```
Cluster Summary:
         Initial_days  Doc_visits        Age  ReAdmis  Overweight
Cluster
0           17.718595    5.002746  53.209226      0.0    0.000000
1           17.291996    5.015743  53.285144      0.0    1.000000
2           63.859507    5.012537  53.940311      1.0    0.704279
```

In [18]:
```
#Cluster visuaization
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)
pca_df = pd.DataFrame(pca_data, columns=['PC1', 'PC2'])
pca_df['Cluster'] = clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='Cluster', palette='Set2', s=60)
plt.title('Patient Clusters (PCA Projection)')
plt.show()
```



In [19]:
```
#Save cleaned dataset
new_data_path = 'output/new_medical_data_kmeans_clusters_D212_I.csv'
df.to_csv(new_data_path, index=False)
```

In [ ]:

In [ ]: